MDRE-LLM: A Tool for Analyzing and Applying LLMs in Software Reverse Engineering

Artur Boronat
School of Computing and Mathematical Sciences
University of Leicester
Leicester, UK
artur.boronat@leicester.ac.uk

Jawad Mustafa
School of Computing and Mathematical Sciences
University of Leicester
Leicester, UK
jm982@student.le.ac.uk

***Abstract*—Understanding and maintaining software systems often requires extracting high-level abstractions, such as domain models, from source code. *MDRE-LLM* addresses this challenge by integrating Large Language Models (LLMs) with traditional Model-Driven Reverse Engineering (MDRE) techniques, offering an innovative approach to automate and enhance domain model recovery. The tool supports flexible granularity strategies and validates LLM-generated models against deterministic baselines.**

***MDRE-LLM* addresses diverse use cases, including analyzing legacy systems with minimal documentation, rapidly comprehending large-scale codebases, and validating LLM performance in reverse engineering tasks. These capabilities have the potential to improve software analysis and refactoring while advance AI-driven research and education by fostering systematic experimentation and collaboration. The tool and a webcast are available at https://zenodo.org/uploads/14072106.**

***Index Terms*—Reverse engineering, domain model recovery, large language models, RAG.**

I. Introduction

Understanding and maintaining complex software systems is one of the most resource-intensive tasks in the software lifecycle, particularly in large, evolving codebases. The challenge lies in recovering high-level abstractions, such as domain models, from low-level source code. A domain model is a high-level abstraction that captures the key entities, relationships, and rules of a problem domain to facilitate shared understanding and guide system development [1]. These abstractions are critical for program comprehension, architectural analysis, and software evolution tasks. Model-Driven Reverse Engineering (MDRE) has emerged as a powerful paradigm for addressing these challenges by extracting conceptual models, such as UML class diagrams, from existing codebases. Traditional MDRE tools rely on static analysis, struggle to capture abstract concepts, and require significant manual intervention [2].

LLMs, trained on vast code and text corpora, have shown ability to understand code semantics and code generation [3], [4], which can be used to generate textual class diagram representations of domain models. In particular, LLMs provide potential opportunities for enhancing MDRE processes by leveraging LLMs to perform tasks that traditionally require human intuition or are beyond the reach of algorithmic approaches [5]. However, applying LLMs to MDRE tasks presents unique challenges. These models often produce hallucinations [6], struggle with large-scale architectural abstractions, and require effective mechanisms for grounding their outputs in the input code. To address these limitations, Retrieval-Augmented Generation (RAG) techniques [7] offer a promising approach, integrating external knowledge sources during generation to improve accuracy and mitigate hallucinations.

In this paper, we introduce *MDRE-LLM*, a MDRE tool that combines LLM capabilities with RAG techniques to automate the recovery of domain models from Java codebases. The tool integrates three levels of granularity for model recovery: *CLASS*, where the LLM processes entire Java files to infer fields, references, operations, and parameters holistically; *COARSE*, which groups elements like fields and method signatures for more structured processing; and *FINE*, where each element—attributes, references, or operations—is extracted individually using focused prompts for higher granularity and accuracy. It features a user-friendly interface for selecting projects, configuring extraction parameters, and visualizing generated UML diagrams. Furthermore, the tool includes a baseline procedural recovery method, which deterministically extracts models using ANTLR, serving as a reference for comparison with LLM-based recovery. This enables empirical evaluation using metrics such as precision, recall, and weighted class complexity (WCC) to assess the fidelity and structural complexity of the generated models. The tool is extensible and, while it provides support for GPT and Gemini models, it can be easily extended to support more types of LLMs. To the best of our knowledge, this is the first MDRE tool that applies RAG-enhanced LLMs for domain model recovery, offering a working environment for studying language models for automating reverse engineering tasks.

The following sections outline use cases, architecture, implementation, and preliminary evaluation.

II. Use Cases and Practical Impact

MDRE-LLM addresses critical challenges in reverse engineering by focusing on two key use cases: (1) analyzing legacy systems and large-scale codebases, and (2) advancing the application of LLMs in reverse engineering.

Legacy systems and large-scale codebases. Legacy systems often suffer from incomplete documentation, making it challenging to understand their architecture and design. Similarly,

modern, large-scale systems require scalable tools for tasks such as developer onboarding and architectural validation. Traditional MDRE tools rely heavily on syntactic analysis and often fail to capture the semantic relationships crucial for understanding complex systems. *MDRE-LLM* addresses these limitations by leveraging RAG to retrieve relevant code fragments, grounding LLM outputs in the source code and mitigating issues like hallucination. This approach enables the recovery of accurate and semantically rich domain models, visualized as UML diagrams to assist in architectural analysis and comprehension.

Advancing LLM-based reverse engineering. The integration of LLMs into reverse engineering represents a promising but underexplored frontier. While LLMs have demonstrated capabilities in generating textual and structural representations of software models, their effectiveness in higher-level tasks like domain model recovery remains uncertain, particularly at the system design level where semantic fidelity is crucial [8]. Current methods lack systematic validation, making it difficult to evaluate the accuracy and completeness of LLM-generated outputs [9]. *MDRE-LLM* provides a systematic framework for evaluating LLM-generated models against deterministic baselines to quantify model fidelity and structural quality.

III. System Architecture and Workflow

MDRE-LLM is a modular framework that leverages LLMs for the reverse engineering of domain models from Java codebases. It integrates semantic capabilities of LLMs with traditional reverse engineering principles to provide a structured approach for extracting, validating, and comparing domain models. The system comprises four core components that implement the workflow to retrieve domain models from Java projects (Java Retrieval, LLM-Based Recoverer, Model Validator, EMF Projector), shown in Fig. 1, and a separate comparison and evaluation component to evaluate the performance of the selected LLM for model recovery.

A. Java Retrieval

The Java Retrieval component (A) preprocesses Java projects to extract the relevant elements required for analysis. It employs the ANTLR framework to parse source files and generate an abstract syntax tree (AST). By traversing the AST, the system identifies key object-oriented constructs, including classes, attributes, references, methods, and annotations. These constructs are normalized by converting them into a standardized format aligned with the domain meta-model, unifying naming conventions, resolving ambiguities, and ensuring consistent data structures for compatibility with LLM prompts and validation. Additional metadata, such as inline comments and annotations, is preserved to provide context for semantic inference by the LLM. Preprocessing simplifies Java files for granularity-based analysis, reducing noise and enhancing recovery efficiency.

B. LLM-Based Recoverer

The LLM-Based Recoverer (B) is the core component of the system, responsible for generating domain pre-models

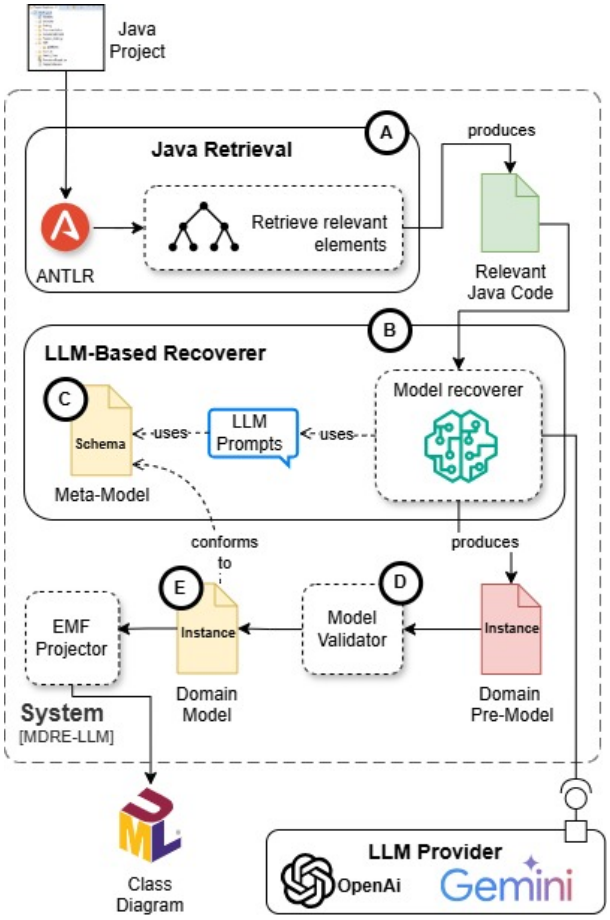

Fig. 1. System Architecture

from processed Java code. It combines context retrieval with generation to ensure accuracy and minimize hallucinations. Using prompts tailored to specific constructs (e.g., classes, attributes, methods), the recoverer guides the LLM in mapping code elements to structured domain entities that conform to the intermediate meta-model (C) of Figure 2. This approach facilitates the integration of new input object-oriented programming languages and the generation of target software model representations. Domain pre-models are represented in a JSON-based format, capturing structured elements such as classes, attributes, references, and methods, with each element adhering to a schema defined by the intermediate meta-model.

The LLM-based Recoverer uses RAG to map code constructs to domain models. The RAG integration enables the tool to retrieve relevant context (e.g., class documentation, annotations, or relationships) from the input Java codebase and use this information as additional input for the LLM.

The recoverer supports three granularity strategies to balance accuracy and computational efficiency: *(1)* the *CLASS* strategy processes entire Java classes in a single prompt, suitable for straightforward cases but prone to overlooking finer details; *(2)* the *COARSE* strategy separates attributes and methods into groups for more focused analysis; and *(3)* the *FINE* strategy processes individual code elements, such as

a single attribute or method, minimizing context for greater precision. These strategies enable systematic analysis of how effectively an LLM processes class information.

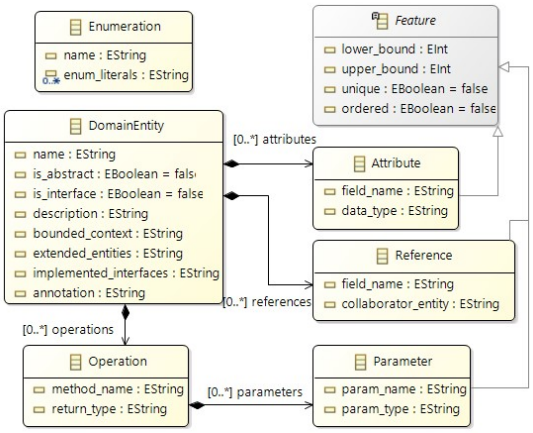

Fig. 2. Intermediate code knowledge representation.

C. Model Validator and Repair

Once the domain pre-model is generated by the LLM-Based Recoverer, it is passed to the Model Validator (D). This component ensures the model conforms to the intermediate meta-model by identifying and resolving inconsistencies. Validation includes detecting undefined classes, incorrect attribute-reference classifications, mismatched opposites in bidirectional associations, and violations of multiplicity constraints, data types, and relationships within the model.

The validator employs rule-based heuristics to automatically repair many inconsistencies in the generated pre-models, such as substituting undefined elements with placeholders, resolving mismatched associations, and converting attributes to references (or vice versa) as needed. These repairs address structural and syntactic issues solely to ensure compatibility with the model comparison tool and do not alter the core semantic content of the LLM-generated models.

D. EMF Projector

The validated domain model is subsequently translated into the Ecore format using the EMF Projector (E). Ecore is a widely adopted meta-modeling standard within the Eclipse Modeling Framework (EMF), enabling seamless integration with downstream engineering tasks such as model transformations, validation, and visualization. The EMF Projector maps the domain entities, attributes, references, and operations to corresponding Ecore constructs, preserving the structural and semantic integrity of the original model.

This projection step ensures that the recovered models are compatible with existing model-driven development pipelines. Additionally, it allows for the generation of UML diagrams and other visual representations, such as PlantUML [1], facilitating system comprehension and analysis.

[1]https://plantuml.com/

E. Comparison and Evaluation Framework

This component evaluates the accuracy and reliability of the LLM-recovered domain models through a two-fold validation process. First, a procedural baseline, custom-implemented using ANTLR, deterministically traverses the Abstract Syntax Tree (AST) to extract structural elements, serving as a reliable and reproducible ground truth. Second, the models are compared using configurable structural and semantic metrics (e.g., precision, recall, and weighted class complexity). This process enables learners and researchers to evaluate model accuracy by observing discrepancies between LLM-derived and baseline models, identify hallucinated or missing elements, and assess the strengths and limitations of AI-driven tools through metrics and visual aids, as illustrated in Fig. 3.

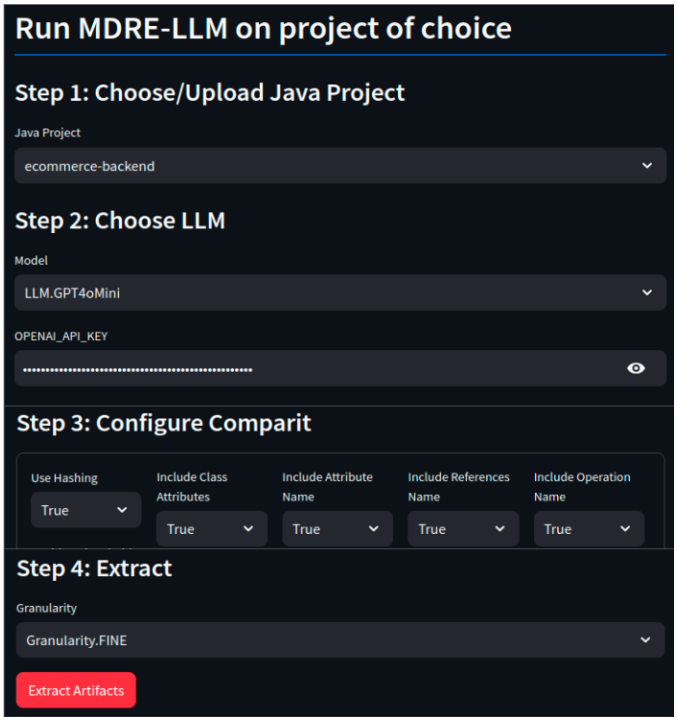

Fig. 3. Configuration.

Structural metrics include precision, recall, and F1-score, which evaluate the completeness and correctness of the recovered elements. Semantic similarity metrics, such as TF-IDF cosine distance, assess the alignment of textual and conceptual elements. Complexity metrics, such as Weighted Class Complexity (WCC), provide insights into the structural fidelity of the models. Differences between the models are visualized to assist in identifying hallucinations, omissions, and inaccuracies in the LLM-recovered model, as shown in Fig. 4. This process provides users with detailed feedback about the models being compared, including a graphical diagrammatic representation, downloadable metrics, Venn diagrams illustrating differences, enabling iterative refinement of the recovery process. The framework also generates reports summarizing the comparison results.

The procedural baseline method was validated through manual inspection to ensure correctness and alignment with domain expectations. Additionally, the effectiveness of the procedural method was analyzed by applying it to Java projects generated from 79 diverse Ecore models in the ModelSet dataset [10] and comparing the recovered models to the original domain models. The method achieved an average precision of 94.75%, recall of 95.35%, F1-score of 94.95%, and a WCC ratio of 98.32%, demonstrating its reliability as a ground truth.

IV. Implementation

In this section, we demonstrate the *MDRE-LLM*'s functionality using an eCommerce application as a running example, highlighting how the implementation supports the workflow from the previous section and the use cases discussed earlier.

Users upload Java projects or select from predefined examples to initiate recovery. For instance, in the case of the eCommerce application, the user selects the project and specifies whether the domain model recovery will be performed using a procedural approach, an LLM, or both for comparison purposes. The interface guides users through subsequent steps, including configuration of the recovery process, granularity settings, and model evaluation.

Once the project is selected, the system preprocesses the Java code using the Java Retrieval component, extracting relevant syntactic elements such as classes, attributes, references, and methods. This normalized representation is then passed to the LLM-Based Recoverer, where domain models are generated using the specified language model. The outputs include detailed Ecore representations of the domain model, visualized as UML diagrams, which are presented to the user along with evaluation metrics.

A. User Interface and Configuration

The tool's interface, implemented in Streamlit, offers an accessible way to configure the domain model recovery process. As shown in Fig. 3, users can specify the granularity level (*CLASS*, *COARSE*, or *FINE*), select an LLM provider (e.g., GPT or Gemini), and provide the necessary API keys. Additionally, users can configure model comparison settings, choosing which model elements (e.g., attributes, references, operations) and properties (e.g., multiplicity bounds, uniqueness) to include in the evaluation.

This configuration flexibility supports a wide range of use cases. For example, in analyzing legacy systems, users can configure the tool to feed detailed, fine-grained input to the LLM, ensuring that every individual code element (such as attributes or methods) is processed independently for greater accuracy. Conversely, for large-scale systems, users can choose coarse-grained recovery, where code elements are grouped to reduce the number of LLM requests, balancing computational efficiency and overall fidelity.

B. Domain Model Recovery and Visualization

Upon configuring the recovery process, the user initiates the model recovery. The interface provides real-time feedback, including logs that detail the LLM's input and output tokens, processing time, and associated costs. This transparency helps users understand the computational resources required for different granularity strategies and LLM configurations.

The tool back-end is built atop Llama-Index [2] and generates the recovered domain model in EMF format and visualizes it as a UML class diagram using PlantUML, making it easier to interpret the results. For instance, in the eCommerce application, the tool successfully identifies core domain entities such as `Order`, `Product`, and `Category`, along with their attributes, references, and operations. These visualizations are essential for understanding system design, especially in poorly documented legacy systems or complex modern codebases.

The RAG-enhanced approach retrieves relevant context, such as annotations or inline comments, from the Java codebase and incorporates it into the LLM's input prompts. This ensures that the generated domain models are grounded in accurate and context-specific information, improving fidelity.

C. Evaluation and Comparison

To validate the accuracy of the recovered models, *MDRE-LLM* includes a procedural recovery method, implemented using ANTLR, that serves as a deterministic baseline for comparison. The baseline systematically extracts structural elements from the AST, ensuring a reliable and reproducible ground truth. Both the LLM-based and baseline models are displayed side-by-side, allowing users to visually and semantically compare their structures. Fig. 4 shows an example of a comparison between the LLM-recovered and baseline models for the eCommerce application. Differences are highlighted, and precision, recall, F1-score, and WCC are computed to assess fidelity and structural quality.

This evaluation identifies hallucinations (e.g., inferred elements not present in the code) and omissions (e.g., missing expected elements) in the recovered models by comparing them against the baseline. Metrics quantify these issues, while visual comparisons, such as the Venn diagrams in Fig. 4, help users spot structural discrepancies efficiently. Green indicates hallucinations, while red shows omissions.

MDRE-LLM was tested on 12 diverse Java projects of varying complexity, including a JavaFX application[3], an API project[4], and an Android application[5]. Two LLMs (*gpt-4o-mini-2024-07-18* and *gemini-1.5-flash*) were compared across three granularity strategies. The *FINE* strategy consistently showed the highest accuracy (F1-score 0.935, WCC ratio 0.929), preserving structural and semantic details, while the *CLASS* strategy had lower fidelity (F1-score 0.811, WCC 0.803) in larger, interconnected codebases. *Gpt-4o-mini-2024-07-18* outperformed *gemini-1.5-flash* in precision and recall but required more processing time, reflecting a trade-off between accuracy and efficiency. These results highlight MDRE-LLM's utility as a framework for systematically evaluating the

[2]https://docs.llamaindex.ai/

[3]https://github.com/yuenci/Java-Car-Rental-System

[4]https://github.com/BT-OpenSource/bt-openlink-java

[5]https://github.com/OpenTracksApp/OpenTracks

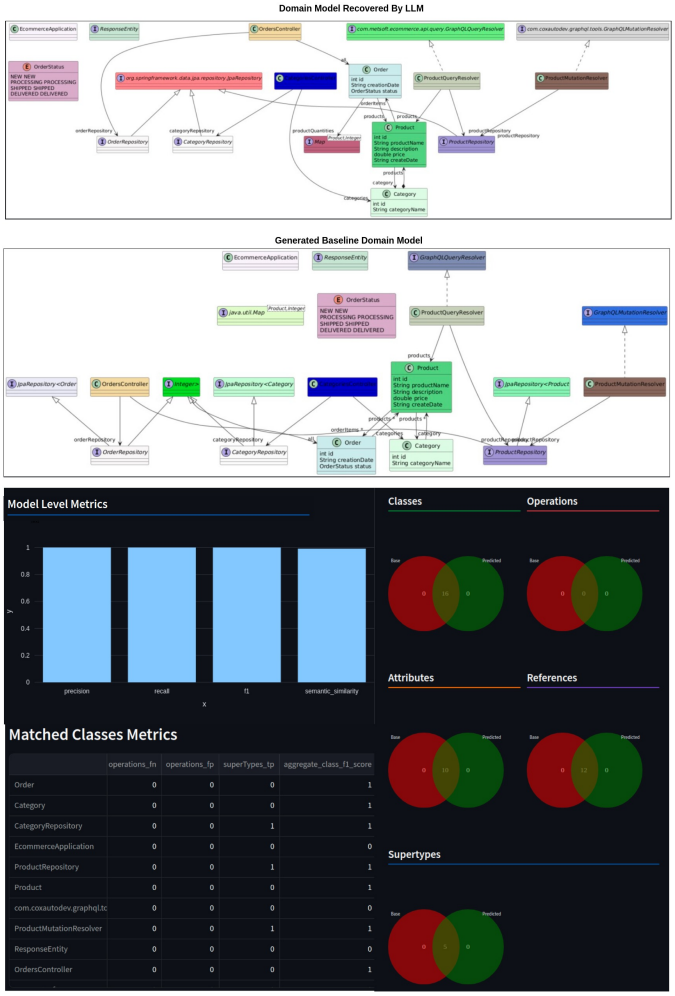

Fig. 4. Model Comparison Results.

performance of LLMs in reverse engineering tasks, providing insights into their effectiveness across diverse project complexities.

V. Related Work

MDRE abstracts complex software systems into high-level representations to support maintenance and evolution. Raibulet et al. [2] identify key challenges, including automation and scalability, in existing MDRE tools. MoDisco [11], a representative MDRE tool, recovers structured, high-level models from legacy codebases using static analysis and the Knowledge Discovery Metamodel (KDM) to support tasks like refactoring and documentation. However, its tight integration with the Eclipse IDE limits scalability for large projects and prevents bulk processing, as projects must be handled individually through the Eclipse UI.

Modular Moose [12] is a flexible meta-modeling framework supporting various programming languages and advanced code analysis. While it emphasizes user-driven exploration and artifact visualization, its reliance on static meta-modeling limits automation capabilities.

Hanan [9] conducted a systematic review of MDRE approaches up to 2023, identifying a reliance on static techniques and a gap in integrating LLMs, presenting a significant opportunity to enhance reverse engineering practices. These limitations highlight the need for novel approaches like MDRE-LLM that improve automation, scalability, and bulk processing.

VI. Conclusions and Future Work

MDRE-LLM integrates LLMs with traditional MDRE techniques to advance the automation and efficiency of domain model recovery. This approach addresses key MDRE challenges, including legacy system analysis, and LLM validation. By combining flexible granularity strategies, model comparison functionality, and reproducible result generation, *MDRE-LLM* offers a platform enabling systematic evaluation of reverse engineering techniques.

Future work on *MDRE-LLM* will focus on expanding its capabilities to support customizable prompts, enabling detailed analysis of LLMs' abstraction abilities for modeling tasks. This customization will provide deeper insights into how LLMs process and transform code into high-level representations, shedding light on their strengths and limitations. Furthermore, exploring the tool's potential in education could provide valuable hands-on learning opportunities.

References

[1] E. Evans, *Domain-Driven Design: Tacking Complexity In the Heart of Software*. Addison-Wesley, 2003.

[2] C. Raibulet, F. Arcelli Fontana, and M. Zanoni, "Model-Driven Reverse Engineering Approaches: A Systematic Literature Review," *IEEE Access*, vol. 5, pp. 14 516–14 542, 2017.

[3] T. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[4] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[5] H. Pearce, B. Tan, P. Krishnamurthy, F. Khorrami, R. Karri, and B. Dolan-Gavitt, "Pop Quiz! Can a Large Language Model Help With Reverse Engineering?" Feb. 2022.

[6] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large Language Models Struggle to Learn Long-Tail Knowledge," Jul. 2023.

[7] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," 2023.

[8] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation," Aug. 2023, arXiv:2308.01861 [cs].

[9] S. Hanan, "Enhancing Model-Driven Reverse Engineering Using Machine Learning," *ICSE 2024 Doctoral Symposium*, 2024.

[10] J. A. H. López, J. L. Cánovas Izquierdo, and J. S. Cuadrado, "ModelSet: a dataset for machine learning in model-driven engineering," *Software and Systems Modeling*, vol. 21, no. 3, pp. 967–986, Jun. 2022.

[11] H. Brunelière, J. Cabot, G. Dupé, and F. Madiot, "MoDisco: A model driven reverse engineering framework," *Information and Software Technology*, vol. 56, no. 8, pp. 1012–1032, Aug. 2014.

[12] N. Anquetil *et al.*, "Modular moose: A new generation of software reverse engineering platform," in *Reuse in Emerging Software Engineering Practices*. Cham: Springer, 2020, pp. 119–134.